Marwadi University	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date: 10/03/2026**	**Enrolment No: 92301733059**

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models. Take the words —-> Give their numeric representation —-> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

Marwadi University	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date: 10/03/2026**	**Enrolment No: 92301733059**

TF(t) = (Number of times term t appears in a document) / (Total number of terms in the document)

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as "I", "a" may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

IDF(t) = log_e(Total number of documents / Number of documents with term t in it)

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus
2. Perform TF vectorization of the corpus

Program (Code):

To be attached with

1. Perform the TF-IDF vectorization of the corpus

Word Embedding

[1] ✓ 0s

```
text = ['i am Sudanese, land of origin',
        'we have more than +240 470 differnts tribe with more than 200 languages.',
'I am retired football player.','now days i am doing camping and traveler.',
'Future is for people they plan it.','Cloud and AI is Gas of Future']
```

[2] ✓ 1s

```
#import the librarries
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer #TF-IDF
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

[3] ✓ 0s

```
#tf
tf_vectorizer = CountVectorizer()
tf_matrix = tf_vectorizer.fit_transform(text)
```

+ Code + Text

[4] ✓ 0s

```
tf_similirty = cosine_similarity(tf_matrix)
```

Marwadi University	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date: 10/03/2026**	**Enrolment No: 92301733059**

```
[3] ✓ 0s
#tf
tf_vectorizer = CountVectorizer()
tf_matrix = tf_vectorizer.fit_transform(text)

[4] ✓ 0s
tf_similirty = cosine_similarity(tf_matrix)

[5] ✓ 0s
print(tf_similirty)

> ... Show hidden output

[6] ✓ 0s
## TF-IDF
tfidf_vectorizer = CountVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(text)

[7] ✓ 0s
tfidf_similirty = cosine_similarity(tfidf_matrix)

[ ]
print(tfidf_similirty)
```

Results:

```
[5] ✓ 0s
print(tf_similirty)

[[1.         0.         0.2236068  0.16903085 0.         0.16903085]
 [0.         1.         0.         0.         0.         0.        ]
 [0.2236068  0.         1.         0.18898224 0.         0.        ]
 [0.16903085 0.         0.18898224 1.         0.         0.14285714]
 [0.         0.         0.         0.         1.         0.28571429]
 [0.16903085 0.         0.         0.14285714 0.28571429 1.        ]]

[6] ✓ 0s
## TF-IDF
tfidf_vectorizer = CountVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(text)

+ Code

[7] ✓ 0s
tfidf_similirty = cosine_similarity(tfidf_matrix)

[ ]
print(tfidf_similirty)

[[1.         0.         0.2236068  0.16903085 0.         0.16903085]
 [0.         1.         0.         0.         0.         0.        ]
 [0.2236068  0.         1.         0.18898224 0.         0.        ]
 [0.16903085 0.         0.18898224 1.         0.         0.14285714]
 [0.         0.         0.         0.         1.         0.28571429]
 [0.16903085 0.         0.         0.14285714 0.28571429 1.        ]]
```

Marwadi University	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date: 10/03/2026**	**Enrolment No: 92301733059**

[6] ✓ 0s
```
## TF-IDF
tfidf_vectorizer = CountVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(text)
```

[7] ✓ 0s
```
tfidf_similirty = cosine_similarity(tfidf_matrix)
```

[9] ✓ 0s
```
print(tfidf_similirty)
```
```
[[1.         0.         0.2236068  0.16903085 0.         0.16903085]
 [0.         1.         0.         0.         0.         0.        ]
 [0.2236068  0.         1.         0.18898224 0.         0.        ]
 [0.16903085 0.         0.18898224 1.         0.         0.14285714]
 [0.         0.         0.         0.         1.         0.28571429]
 [0.16903085 0.         0.         0.14285714 0.28571429 1.        ]]
```

[8] ✓ 0s
```
#Doc Simil Func
def doc_sim(similarity_matrix, text):
  for i in range(len(text)):
    sim = similarity_matrix[i].copy()
    sim[i]=-1
    top_idx = np.argmax(sim)
    print(text[i], "-->" , text[top_idx])
```

[10] ✓ 0s
```
doc_sim(similarity_matrix=tfidf_similirty, text=text)
```
```
i am Sudanese, land of origin --> I am retired football player.
we have more than +240 470 differnts tribe with more than 200 languages. --> i am Sudanese, land of origin
I am retired football player. --> i am Sudanese, land of origin
now days i am doing camping and traveler. --> I am retired football player.
Future is for people they plan it. --> Cloud and AI is Gas of Future
Cloud and AI is Gas of Future --> Future is for people they plan it.
```

Observation:

1. **BoW:** Simply counts occurrences. It often yields higher similarity scores because common words (I, am, a) pull the vectors together, even if the "unique" meaning is different.
2. **TF:** Normalizes the BoW by document length. While it prevents long documents from dominating, it still treats common stop words as highly important.
3. **TF-IDF :** Usually provides the most accurate "semantic" similarity. It penalizes common words like "cat" (if it appears in every document) and rewards rare words like "Steve" or "regularly," highlighting what actually makes a document unique.

Marwadi University	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date: 10/03/2026**	**Enrolment No: 92301733059**

Post Lab Exercise:

Take three documents and find similarity using

a. BoW vectorization

b. TF Vectorization

c. TF-IDF Vectorization

[2] ✓ 0s

```
documents = [
    "The cat sat on the mat.",
    "The dog ran in the park.",
    "The cat and dog played together."
]
print("Original Documents:")
for i, doc in enumerate(documents):
    print(f"Document {i+1}: {doc}")
```

```
Original Documents:
Document 1: The cat sat on the mat.
Document 2: The dog ran in the park.
Document 3: The cat and dog played together.
```

```
# BoW Vectorization (binary representation)
bow_vectorizer = CountVectorizer(binary=True)
bow_matrix = bow_vectorizer.fit_transform(documents)

print("BoW Matrix:")
print(bow_matrix.toarray())
print("Features (vocabulary):")
print(bow_vectorizer.get_feature_names_out())

bow_similarity = cosine_similarity(bow_matrix)
print("\nBoW Similarity Matrix:")
print(bow_similarity)
```

Marwadi University	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date: 10/03/2026**	**Enrolment No: 92301733059**

```
BoW Matrix:
[[0 1 0 0 1 1 0 0 0 1 1 0]
 [0 0 1 1 0 0 1 0 1 0 1 0]
 [1 1 1 0 0 0 0 1 0 0 1 1]]
Features (vocabulary):
['and' 'cat' 'dog' 'in' 'mat' 'on' 'park' 'played' 'ran' 'sat' 'the'
 'together']

BoW Similarity Matrix:
[[1.         0.2        0.36514837]
 [0.2        1.         0.36514837]
 [0.36514837 0.36514837 1.        ]]
```

```
# TF Vectorization (term frequency)
tf_vectorizer = CountVectorizer()
tf_matrix = tf_vectorizer.fit_transform(documents)

print("TF Matrix:")
print(tf_matrix.toarray())
print("Features (vocabulary):")
print(tf_vectorizer.get_feature_names_out())

tf_similarity = cosine_similarity(tf_matrix)
print("\nTF Similarity Matrix:")
print(tf_similarity)
```

```
TF Matrix:
[[0 1 0 0 1 1 0 0 0 1 2 0]
 [0 0 1 1 0 0 1 0 1 0 2 0]
 [1 1 1 0 0 0 0 1 0 0 1 1]]
Features (vocabulary):
['and' 'cat' 'dog' 'in' 'mat' 'on' 'park' 'played' 'ran' 'sat' 'the'
 'together']

TF Similarity Matrix:
[[1.         0.5        0.4330127]
 [0.5        1.         0.4330127]
 [0.4330127 0.4330127 1.        ]]
```

Marwadi University	Marwadi University Faculty of Engineering and Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	**Aim:** To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	**Date: 10/03/2026**	**Enrolment No: 92301733059**

```
# TF-IDF Vectorization
tfidf_vectorizer_new = TfidfVectorizer()
tfidf_matrix_new = tfidf_vectorizer_new.fit_transform(documents)

print("TF-IDF Matrix:")
print(tfidf_matrix_new.toarray())
print("Features (vocabulary):")
print(tfidf_vectorizer_new.get_feature_names_out())

tfidf_similarity_new = cosine_similarity(tfidf_matrix_new)
print("\nTF-IDF Similarity Matrix:")
print(tfidf_similarity_new)
```

```
TF-IDF Matrix:
[[0.         0.34101521 0.         0.         0.44839402 0.44839402
  0.         0.         0.         0.44839402 0.52965746 0.        ]
 [0.         0.         0.34101521 0.44839402 0.         0.
  0.44839402 0.         0.44839402 0.         0.52965746 0.        ]
 [0.4711101  0.35829137 0.35829137 0.         0.         0.
  0.         0.4711101  0.         0.         0.27824521 0.4711101 ]]
Features (vocabulary):
['and' 'cat' 'dog' 'in' 'mat' 'on' 'park' 'played' 'ran' 'sat' 'the'
 'together']

TF-IDF Similarity Matrix:
[[1.         0.28053703 0.26955746]
 [0.28053703 1.         0.26955746]
 [0.26955746 0.26955746 1.        ]]
```

Comment over the answer obtained in each of the cases.

The TF-IDF Similarity Matrix assigns weights to words based on their frequency in a document and their inverse frequency across all documents, giving more importance to unique words. For example:

- The similarities for TF-IDF are generally lower than TF. This is because common words like 'the' (which appears in all documents) are down-weighted by the IDF component, while less common but more distinguishing words ('cat', 'dog', 'mat', 'park', 'played') contribute more meaningfully to the similarity score.
- Document 1 and Document 2 have a similarity of 0.280, while Document 1 and Document 3 have 0.269, and Document 2 and Document 3 also have 0.269.
- The differences in similarity values across BoW, TF, and TF-IDF highlight how each method handles word importance: BoW only cares about presence, TF about frequency, and TF-IDF about discriminatory power of words.